

Dalla chat all'agente

Function calling, MCP e il ciclo osserva-ragiona-agisci: cosa cambia quando il modello smette di rispondere e comincia a chiedere di fare.

Il limite di tutto il percorso fin qui: il modello genera solo testo

- Con il RAG il modello legge meglio, ma l'output resta lo stesso: una sequenza di token.
- «Ordina 50 pezzi» produce una frase che descrive l'ordine, non un ordine.
- Nessuna chiamata HTTP, nessuna scrittura su file, nessuna transazione: il motore non ha effetti collaterali.
- Il salto non è rendere il modello più potente: è dare un significato eseguibile a parte del suo testo.

IL PERCORSO FIN QUI

M1 previsione probabilistica

M2 token, embedding, finestra

M3 dati aziendali in lettura

Tre moduli, un solo tipo di output: testo.

Il salto di paradigma: function calling / tool use

```
"tools": [{ "name": "get_giacenza ", "description":  
"Giacenza di un articolo per codice", "parameters":  
{ "codice": {"type": "string"} } } ] ← risposta del  
modello {"tool": "get_giacenza", "codice":  
"MAT-4471/B"}
```

- Nel payload si aggiunge l'elenco delle funzioni disponibili, con nome, descrizione e parametri.
- Il modello, invece di rispondere in prosa, produce un JSON che dice quale funzione vuole e con quali argomenti.
- Meccanicamente non è cambiato niente: è sempre previsione di token, solo vincolata a un formato.
- Il modello non esegue e non sa se la funzione esiste: propone una chiamata.

Chi esegue davvero l'azione: un livello di codice, non il modello

IL MODELLO

Emette una richiesta in JSON. Nessuna credenziale, nessuna connessione, nessun permesso.

L'ORCHESTRATORE

Valida i parametri, applica i permessi, chiama l'API reale, restituisce il risultato al modello.

- È lo stesso strato che nel modulo 1 costruiva il prompt: qui diventa anche il punto di controllo delle azioni.
- Il JSON che arriva va trattato come input non fidato: schema, whitelist, controllo dei valori. Sempre.
- Un agente non ha più poteri di quelli che gli concede questo livello: il perimetro lo decidete voi.

Lo standard di oggi per collegare agente e tool: MCP (Model Context Protocol)

- Senza standard ogni integrazione è ad hoc: N modelli $\times$ M sistemi, tutti da riscrivere a mano.
- MCP definisce come un server dichiara i tool disponibili e come un client li invoca: uno schema, non una libreria.
- Il tool si scrive una volta e vale per client diversi: assistente in IDE, chat interna, agente batch.
- Per un'infrastruttura aziendale è un servizio come gli altri: processo, endpoint, log, autorizzazioni.

I TOOL DELLA DEMO

```
get_giacenza(codice)
```

```
get_ordini_produzione(stato)
```

```
scrivi_report(titolo, testo)
```

Due in lettura, uno in scrittura: la differenza conta per i permessi.

Il ciclo dell'agente: osserva → ragiona → agisce → osserva il risultato

1 · OSSERVA

La finestra contiene la richiesta e i risultati dei passi precedenti.

2 · RAGIONA

Il modello scrive: rispondere, oppure chiamare un tool.

3 · AGISCE

L'orchestratore esegue la chiamata sul sistema reale.

4 · OSSERVA

Il risultato torna nella finestra e il ciclo riparte.

- Il loop sta nel vostro codice: il modello resta stateless e viene richiamato da zero a ogni giro.
- Il ciclo termina quando il modello risponde in prosa invece di chiedere un altro tool.

Incatenare più tool per una singola richiesta

```
richiesta"Verifica se gli ordini aperti sono coperti dalle giacenze e scrivimi un report" passo  
1get_ordini_produzione(stato="aperto") → 4 ordini passo 2get_giacenza("MAT-4471/B") → 37 pz passo  
3get_giacenza("MAT-8820") → 0 pz passo 4scrivi_report("Copertura ordini", "...") → ok
```

- La sequenza non è codificata da nessuna parte: emerge a runtime, un passo alla volta.
- Ogni risultato allunga la finestra: costo e latenza crescono a ogni passo, come nel modulo 2.
- Differenza sostanziale da un workflow: due esecuzioni della stessa richiesta possono seguire strade diverse.
- L'ultimo passo scrive: da qui in poi l'errore non è più solo una risposta sbagliata.

Perché serve un limite di iterazioni

- Niente garantisce che il ciclo si chiuda: il modello può richiedere lo stesso tool all'infinito.
- Cause tipiche: un tool che risponde con un errore poco chiaro, un parametro sbagliato, un obiettivo irraggiungibile.
- Ogni giro costa una chiamata al modello e una al sistema: un loop non sorvegliato è un problema di capacity, non solo di qualità.
- Il tetto va nell'orchestratore, insieme a budget di token e timeout: gli stessi presidi di qualsiasi retry.

GUARDIE DEL LOOP

```
max_iterations = 8  
max_tokens_totali = 12_000  
timeout_per_tool = 5s  
  
on_limit → stop + messaggio  
all'utente + log del ciclo
```

Rendere visibile il ragionamento: ogni chiamata e risultato, non solo la risposta finale

SOLO RISPOSTA FINALE

«Il report è pronto: due ordini non sono coperti.»

Se il numero è sbagliato non si sa dove: tool, parametri o generazione.

CICLO ESPOSTO

```
get_giacenza("MAT-8820") → {"pz": 0, "dep": "B2"}
```

Il difetto si localizza al passo esatto in cui è nato.

- Come lo switch del modulo 3, ma qui i passi sono azioni: la traccia serve anche come registro di audit.
- Minimo da loggare per ogni passo: tool, argomenti, esito, durata, utente che ha originato la richiesta.

Il payload reale inviato ad ogni passo del ciclo

```
{ "messages": [ {"role": "system", "content": "..."},  
{"role": "user", "content": "Verifica..."},  
{"role": "assistant", "tool_call":  
{"name": "get_ordini_produzione"}}, {"role": "tool"  
", "content": "[4 ordini]"},  
{"role": "assistant", "tool_call":  
{"name": "get_giacenza"}}, {"role": "tool"  
", "content": "{pz:37}" } ], "tools": [ ... ] }
```

- Stessa POST del modulo 2, due campi in più: l'elenco dei tool e i messaggi di ruolo tool.
- Il risultato di un tool non è codice eseguito nel modello: rientra come testo nella finestra.
- Il payload si allunga a ogni passo: al giro 6 il modello rilegge tutta la storia del ciclo.
- Qui si vede in chiaro cosa esce dalla rete aziendale: descrizioni dei tool e dati restituiti, inclusi.

Non tutti i modelli sono ugualmente affidabili nel tool-calling

- Il tool-calling si impara in addestramento: un modello non addestrato a farlo lo simula male, in prosa.
- Errori tipici dei modelli piccoli: JSON malformato, parametri inventati, tool inesistenti, un solo passo invece di tre.
- Contromisure lato codice: validazione dello schema, retry con l'errore rispedito al modello, descrizioni brevi e non ambigue.
- Meno tool esposti e nomi più espliciti: la scelta migliora più con questo che con un modello più grande.

RISPOSTA NON VALIDA

Chiamo get_stock per verificare la giacenza...

Nessun JSON e un tool che non esiste:
l'orchestratore deve rifiutare, non interpretare.

Oltre il singolo agente: sistemi multi-agente (accenno)

COME FUNZIONA

Un coordinatore spezza il compito e lo passa ad agenti con pochi tool ciascuno: magazzino, produzione, reportistica.

Ogni agente ha finestra e permessi propri: contesti più corti e più mirati.

COSA COSTA

Più cicli, più chiamate, più punti in cui un errore si propaga senza essere notato.

Ha senso quando un solo agente satura la finestra o mescola permessi diversi.

Per un primo progetto interno: un agente, pochi tool, ciclo visibile. Il multi-agente è un'ottimizzazione successiva.

Verso il modulo 5: un agente decide da solo — e se il testo che riceve è scritto con cattive intenzioni?

IL PRESUPPOSTO

L'agente sceglie i tool leggendo testo. Istruzioni e dati arrivano nello stesso canale: la finestra.

IL RISCHIO

Un documento recuperato dal RAG può contenere una frase scritta per farsi obbedire.

- Nel modulo 3 un input malevolo produceva una risposta sbagliata; ora può produrre un'azione.
- Il perimetro non è più solo la rete: è l'insieme dei tool esposti e dei permessi con cui girano.
- Modulo 5 — Sicurezza: prompt injection, dati che escono, e cosa mettere davanti a un agente in produzione.